PHASE 6 — DEEP DIVE

Local LLM Inference & Modern Deployment Stack

Ollama Production · Quantisation · vLLM · Ray Serve · Modal · Replicate · BentoML · Triton

Month 18–20 · Every resource with exact video title + channel name

Phase Duration	12 weeks (Month 18, 19, and 20 of the full roadmap)
Daily Commitment	3–4 hours weekdays, 5–6 hours weekends
Your Starting Point	Phases 1–5 complete — production AI SaaS with RAG, agents, evals, and cost optimisation live on Railway
Goal of This Phase	Own the model serving layer — run your own LLMs at production scale without depending entirely on OpenAI/Anthropic APIs
Core Question	Why pay $0.0025/request to OpenAI when you can serve your own Llama 3.1 70B at $0.0002/request? Phase 6 answers this question with production infrastructure.
What Changes	You stop being a pure API consumer and become a model infrastructure engineer. You control the hardware, the serving framework, the autoscaling, and the cost model.
Total Topics	9 core topics — quantisation, Ollama production, vLLM, OpenAI-compatible servers, Ray Serve, Modal, Replicate, BentoML, Triton Inference Server
Capstone Project	Self-Hosted LLM Inference Cluster — Portfolio Project #6

⚡ Phase 6 Philosophy — Control the Model Serving Layer

Every previous phase built on top of someone else's model serving infrastructure — you called OpenAI's API and paid their rates. Phase 6 changes that. You will learn to host and serve open-source models yourself, with production-grade throughput, autoscaling, and monitoring. This is the infrastructure layer that separates mid-level AI engineers from senior ones.

Decision	API-Only Approach	Phase 6 Approach
Cost at 10M tokens/day	~$25,000/month (GPT-4o)	~$2,000/month (self-hosted Llama 3.1 70B on A100)
Data privacy	Data leaves your infrastructure	Data never leaves your VPC — critical for enterprise
Latency	50–200ms + network	10–50ms — model is in your network
Model control	No fine-tuning flexibility at inference	Deploy your own fine-tuned models instantly
Vendor lock-in	Complete dependency on one provider	Switch models without changing application code
Downtime risk	OpenAI outages take your product down	Your infra — your SLAs — your redundancy
Compliance	Limited — data goes to US servers	Deploy in any region, on-prem, or air-gapped

Phase 6 Target Outcome: By the end of Phase 6, you can take any HuggingFace model and serve it at production scale with proper autoscaling, health checks, and an OpenAI-compatible API. Your existing application code needs zero changes to switch from OpenAI to your self-hosted model — only an environment variable changes.

TOPIC 1 — Model Quantisation (GGUF, GPTQ, AWQ, INT8, FP16) · *Weeks 1–2 · Run a 70B model on hardware you can actually afford*

Quantisation is the process of reducing the numerical precision of model weights — from 32-bit or 16-bit floats to 8-bit or 4-bit integers. The result: models 2–8x smaller in memory with 5–10% quality loss at 4-bit, and nearly zero quality loss at 8-bit. This is what makes running large models on consumer hardware possible.

1.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Quantisation Explained — GGUF, GPTQ, AWQ, INT4, INT8 — Which to Use	Fahd Mirza	YouTube → Search 'Quantisation Explained GGUF GPTQ AWQ INT4 INT8 Fahd Mirza'	The definitive quantisation comparison video. Watch first — sets context for all other resources.
2	Video	GGUF Format Explained — How Ollama and llama.cpp Use It	Matt Williams (Ollama creator)	YouTube → Search 'GGUF Format Explained Ollama llama.cpp Matt Williams'	From the Ollama creator — explains GGUF internals, quantisation levels (Q2_K to Q8_0).
3	Video	AWQ Quantisation — 4-Bit Activation-Aware Weight Quantisation	Trelis Research	YouTube → Search 'AWQ Quantisation 4-Bit Activation-Aware Weight Trelis Research'	AWQ is the highest-quality 4-bit method. Best explanation of why AWQ beats GPTQ at 4-bit.
4	Video	GPTQ Quantisation — Quantise Any Model to 4-bit in Python	Trelis Research	YouTube → Search 'GPTQ Quantisation Quantise Model 4-bit Python Trelis Research'	Hands-on: quantise a model from HuggingFace to GPTQ 4-bit using AutoGPTQ library.
5	Video	BitsAndBytes INT8 Quantisation — Load Large Models on Small GPUs	Abhishek Thakur	YouTube → Search 'BitsAndBytes INT8 Quantisation Large Models Small GPUs Abhishek Thakur'	BitsAndBytes INT8 for inference — load a 13B model on a single GPU with minimal quality loss.
6	Video	llama.cpp — Run LLMs Locally Without GPU (CPU Inference)	Andrei Dumitrescu (Crystal Mind Academy)	YouTube → Search 'llama.cpp Run LLMs Locally Without GPU CPU Inference Andrei Dumitrescu'	llama.cpp is the foundation of Ollama. Understand it to debug Ollama performance issues.
7	Video	Comparing Quantisation Levels — Q4_K_M vs Q5_K_M vs Q8_0 — Quality vs Speed	Fahd Mirza	YouTube → Search 'Comparing Quantisation Levels Q4 Q5 Q8 Quality Speed Fahd Mirza'	Side-by-side quality benchmarks across GGUF quantisation levels — choose the right one.
8	Video	How to Choose the Right Model Quantisation for Production	Sam Witteveen	YouTube → Search 'Choose Right Model Quantisation Production Sam Witteveen'	Decision framework: when to use INT4 vs INT8 vs FP16 — production latency vs memory trade-offs.

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
9	Docs	llama.cpp GitHub Wiki — Quantisation Guide	llama.cpp team	github.com/ggerganov/llama.cpp/wiki — read Quantisation section completely	The authoritative reference on GGUF quantisation levels and their memory/quality characteristics.

1.2 — Quantisation Formats Deep Dive

The Four Main Formats — What Each Is and When to Use It

GGUF (llama.cpp format)	The universal format — runs on CPU, Apple Silicon, and GPU. Used by Ollama, LM Studio, GPT4All. 12+ quantisation levels from Q2_K (tiny, low quality) to Q8_0 (near-FP16 quality). BEST FOR: Ollama, local dev, edge deployment.
GPTQ (GPU-optimised)	Quantises weights using second-order Hessian information — smarter than round-to-nearest. Requires GPU for inference. Used by AutoGPTQ and ExLlamaV2. BEST FOR: GPU servers running PyTorch-based inference.
AWQ (Activation-Aware)	Identifies which weights are most important by examining activations — preserves them at higher precision. Higher quality than GPTQ at the same bit-width. BEST FOR: 4-bit production inference where quality matters most.
BitsAndBytes (INT8/NF4)	Runtime quantisation — no need to pre-quantise. Load a model in INT8 or NF4 on-the-fly. Used in QLoRA fine-tuning (Phase 4). Slower than GPTQ/AWQ at inference. BEST FOR: fine-tuning, one-off inference on limited hardware.

GGUF Quantisation Level Decision Table

GGUF Level	Memory (7B model)	Quality vs FP16	Best Use Case
Q2_K	~2.8 GB	Significant degradation — 8–15% quality loss	Extremely RAM-limited — last resort
Q3_K_M	~3.3 GB	Noticeable degradation — 5–8% quality loss	Memory constrained, quality not critical
Q4_K_M	~4.1 GB	Minor degradation — 2–4% quality loss	★BEST DEFAULT: great balance of size and quality
Q5_K_M	~4.8 GB	Minimal degradation — 1–2% quality loss	More RAM available, want near-full quality
Q6_K	~5.5 GB	Barely perceptible — <1% quality loss	High RAM, maximum quality in GGUF format
Q8_0	~7.0 GB	Essentially FP16 quality — <0.1% loss	Reference quality — use when quality is paramount

GGUF Level	Memory (7B model)	Quality vs FP16	Best Use Case
FP16	~14.0 GB	Baseline — full quality	Production GPU inference (vLLM, etc.) on capable hardware

Use Q4_K_M as your default for development and testing. Use FP16 for production vLLM serving (it handles quantisation internally with AWQ). Never use Q2_K in production — the quality degradation is user-visible. The sweet spot is Q4_K_M: 4.1 GB for a 7B model, great quality, fast inference.

1.3 — Hands-On: Quantise a Model

```
# Option 1: Download a pre-quantised GGUF from HuggingFace (easiest)
# Most popular models have pre-quantised GGUF versions on the Hub
# Search: 'TheBloke/Llama-3-8B-Instruct-GGUF' on huggingface.co

# Option 2: Quantise a model yourself with llama.cpp
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp && make -j4

# Download original FP16 model from HuggingFace
huggingface-cli download meta-llama/Llama-3.1-8B-Instruct \
  --local-dir ./models/llama3-fp16

# Convert to GGUF format
python convert_hf_to_gguf.py ./models/llama3-fp16 \
  --outfile ./models/llama3-f16.gguf --outtype f16

# Quantise to Q4_K_M (the default recommended level)
./llama-quantize ./models/llama3-f16.gguf \
  ./models/llama3-q4km.gguf Q4_K_M

# Result: 14GB FP16 → 4.1GB Q4_K_M
# Run it with llama.cpp directly:
./llama-cli -m ./models/llama3-q4km.gguf -p 'Hello world'

# Or import into Ollama:
# Create Modelfile:
# FROM ./models/llama3-q4km.gguf
# PARAMETER num_ctx 4096
ollama create llama3-custom -f Modelfile
ollama run llama3-custom
```

1.4 — Stop Conditions

- ☐ Explain the difference between GGUF, GPTQ, AWQ, and BitsAndBytes — when to use each

☐ Download a Q4_K_M GGUF model from HuggingFace and run it with llama.cpp CLI
☐ Quantise a model yourself: FP16 HuggingFace model → Q4_K_M GGUF using llama.cpp
☐ Compare quality: same 20 prompts on Q4_K_M vs Q8_0 vs FP16 — document the differences
☐ Import a custom GGUF into Ollama — serve it as a named model with a custom Modelfile
☐ Memory calculation: predict RAM needed for Llama 3.1 70B at Q4_K_M before downloading

TOPIC 2 — Ollama in Production · *Week 2 · Local inference at scale with OpenAI-compatible API*

Ollama is not just a local dev tool — it is a production-ready model server with an OpenAI-compatible API, GPU acceleration, model caching, concurrent request handling, and a REST API. For small-to-medium AI applications running on a single GPU server, Ollama is the simplest path to self-hosted LLM inference.

2.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Ollama in Production — Deploy to a Server with GPU	Matt Williams (Ollama creator)	YouTube → Search 'Ollama Production Deploy Server GPU Matt Williams'	From the creator — exactly how to run Ollama in production on a remote GPU server.
2	Video	Ollama REST API — Build Applications with Ollama	Matt Williams (Ollama creator)	YouTube → Search 'Ollama REST API Build Applications Matt Williams'	Full Ollama API tutorial — /api/generate, /api/chat, /api/embeddings, /api/ps endpoints.
3	Video	Ollama Docker Deployment — Run Ollama in a Container	Mervin Praison	YouTube → Search 'Ollama Docker Deployment Run Ollama Container Mervin Praison'	Containerise Ollama for repeatable deployment on any cloud provider.
4	Video	Ollama GPU Setup — NVIDIA CUDA and AMD ROCm Configuration	Fahd Mirza	YouTube → Search 'Ollama GPU Setup NVIDIA CUDA AMD ROCm Fahd Mirza'	Configure GPU acceleration for Ollama — critical for production performance.
5	Video	Ollama Modelfile — Custom System Prompts, Parameters, and Templates	Matt Williams (Ollama creator)	YouTube → Search 'Ollama Modelfile Custom System Prompts Parameters Matt Williams'	Create custom models with baked-in system prompts — deploy your fine-tuned models as named models.
6	Video	Multiple Models with Ollama — Concurrent Model Loading	Mervin Praison	YouTube → Search 'Multiple Models Ollama Concurrent Model Loading Mervin Praison'	Run multiple models simultaneously — route different tasks to different models.
7	Video	Ollama Performance Tuning — Context Size, Threading, and Batching	Fahd Mirza	YouTube → Search 'Ollama Performance Tuning Context Size Threading Batching Fahd Mirza'	OLLAMA_NUM_PARALLEL, OLLAMA_MAX_LOADED_MODELS, num_ctx — tune for throughput vs latency.
8	Video	Open WebUI with Ollama — Production Web Interface Setup	Mervin Praison	YouTube → Search 'Open WebUI Ollama Production Web Interface Setup Mervin Praison'	Deploy Open WebUI as your team's self-hosted ChatGPT — with user auth and model selection.

2.2 — Ollama Production Architecture

Production Ollama Setup on a GPU Server

```
# Deploy Ollama on a GPU server (e.g., Lambda Labs A10G, $0.75/hr)
# SSH into server, then:

# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Configure for production (environment variables)
sudo tee /etc/systemd/system/ollama.service.d/override.conf << EOF
[Service]
Environment='OLLAMA_HOST=0.0.0.0:11434'          # listen on all interfaces
Environment='OLLAMA_NUM_PARALLEL=4'               # 4 concurrent requests
Environment='OLLAMA_MAX_LOADED_MODELS=3'          # keep 3 models in VRAM
Environment='OLLAMA_KEEP_ALIVE=30m'               # keep models warm for 30 min
EOF

sudo systemctl daemon-reload && sudo systemctl restart ollama

# Pull models (runs in background, check with 'ollama ps')
ollama pull llama3.1:8b-instruct-q4_K_M
ollama pull nomic-embed-text
ollama pull qwen2.5-coder:7b-instruct-q4_K_M

# Test — OpenAI-compatible API
curl http://YOUR_SERVER_IP:11434/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -d '{"model": "llama3.1:8b-instruct-q4_K_M",
       "messages": [{"role": "user", "content": "Hello"}]}'

# Your existing OpenAI SDK code works with ONE change:
# OPENAI_BASE_URL=http://YOUR_SERVER_IP:11434/v1
# OPENAI_API_KEY=ollama  (any string, not used by Ollama)
```

Ollama Key Environment Variables for Production

OLLAMA_HOST	Set to 0.0.0.0:11434 to accept connections from other machines (default: localhost only)
OLLAMA_NUM_PARALLEL	Max concurrent request processing. Default: 1. Set to 4 for production throughput.
OLLAMA_MAX_LOADED_MODELS	How many models to keep in VRAM simultaneously. Default: 1. Set to 3 if VRAM allows.
OLLAMA_KEEP_ALIVE	How long to keep a model in memory after last request. Default: 5m. Set to 30m for prod.

OLLAMA_FLASH_ATTENTION	Set to 1 to enable Flash Attention — 20–40% faster inference on supported hardware.
OLLAMA_GPU_OVERHEAD	Reserve VRAM for system use (bytes). Prevents OOM errors when loading models.

For a single A10G GPU (24GB VRAM) in production: run Llama 3.1 8B (Q4_K_M, ~4.5GB) + nomic-embed-text (~280MB) + one backup model simultaneously. Set OLLAMA_NUM_PARALLEL=8 — the 8B model serves 8 requests concurrently with good throughput. Cost: ~$0.54/hr on Lambda Labs for 24/7 operation.

2.3 — Ollama vs vLLM: When to Use Each

Factor	Ollama	vLLM
Best for	Single-server, small team, ease of setup	High-throughput, multi-GPU, enterprise scale
Setup time	5 minutes — one curl command	30–60 minutes — Docker, CUDA drivers, config
Throughput	~50–100 req/min on A10G (8B model)	~500–2000 req/min on A10G (8B model, PagedAttention)
Quantisation	GGUF natively — Q4_K_M, Q5_K_M, Q8_0	AWQ, GPTQ, FP16, BF16 — not GGUF
Multi-GPU	Limited — single GPU focus	Native multi-GPU tensor parallelism
Model switching	Instant — 'ollama pull' and run	Restart server per model (or use multiple endpoints)
OpenAI compatible	Yes — /v1/chat/completions, /v1/embeddings	Yes — identical API surface
Memory efficiency	Good — GGUF quantisation reduces memory	Excellent — PagedAttention maximises GPU utilisation
Monitoring	Basic — /api/ps endpoint	Rich — Prometheus metrics, detailed throughput stats
Cost threshold	Best under ~100 req/min sustained load	Better above ~100 req/min sustained load

2.4 — Stop Conditions

- ☐ Deploy Ollama on a cloud GPU server — accessible via public IP, HTTPS via Nginx reverse proxy
- ☐ OpenAI SDK connects to your Ollama server — zero code changes except OPENAI_BASE_URL
- ☐ OLLAMA_NUM_PARALLEL=4 — benchmark: confirm 4 concurrent requests complete faster than sequential
- ☐ Custom Modelfile: deploy your Phase 4 fine-tuned model as a named Ollama model
- ☐ Open WebUI: team members can chat with your Ollama models via browser without API keys

- ☐ Nginx reverse proxy: Ollama behind HTTPS with basic API key authentication header validation

TOPIC 3 — vLLM: High-Throughput LLM Serving · *Weeks 3–4 · PagedAttention — 24x more throughput than Hugging Face pipelines*

vLLM is the most important open-source LLM inference engine. Its key innovation — PagedAttention — manages GPU memory for KV caches like an OS manages RAM, eliminating memory waste and enabling near-perfect GPU utilisation. The result: 24x higher throughput than naive HuggingFace inference. Every serious AI infrastructure engineer uses vLLM.

3.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	vLLM Explained — PagedAttention and High-Throughput LLM Serving	Andrej Karpathy (referenced in talks)	YouTube → Search 'vLLM Explained PagedAttention High Throughput LLM Serving'	Best conceptual explanation of vLLM and PagedAttention. Watch before hands-on.
2	Video	vLLM Tutorial — Deploy Any HuggingFace Model with OpenAI-Compatible API	Trelis Research	YouTube → Search 'vLLM Tutorial Deploy HuggingFace Model OpenAI Compatible API Trelis Research'	Hands-on vLLM deployment — from HuggingFace model to serving endpoint in <15 minutes.
3	Video	vLLM Offline Inference — Batch Processing with vLLM	Trelis Research	YouTube → Search 'vLLM Offline Inference Batch Processing Trelis Research'	Offline inference mode for bulk document processing — embed 1M docs in hours.
4	Video	vLLM with AWQ Quantisation — Serve 70B Models on a Single A100	Fahd Mirza	YouTube → Search 'vLLM AWQ Quantisation Serve 70B Models Single A100 Fahd Mirza'	Run Llama 3.1 70B on a single A100 (80GB) using AWQ 4-bit — real performance benchmarks.
5	Video	vLLM Continuous Batching — How It Works and Why It Matters	Anyscale (official channel)	YouTube → Anyscale YouTube → Search 'vLLM Continuous Batching How It Works'	Continuous batching is vLLM's throughput secret — understand it before benchmarking.
6	Video	vLLM vs TGI vs Triton — Which LLM Serving Framework Should You Use	Sam Witteveen	YouTube → Search 'vLLM vs TGI vs Triton LLM Serving Framework Sam Witteveen'	Full comparison of all major inference servers — TGI (HuggingFace), vLLM, Triton.
7	Video	vLLM Multi-GPU Serving — Tensor Parallelism for Large Models	vLLM (official channel)	YouTube → vLLM official YouTube → Search 'vLLM Multi-GPU Serving Tensor Parallelism'	Serve 70B models across multiple GPUs using tensor parallelism — production multi-GPU setup.
8	Video	vLLM Production Deployment — Docker, Kubernetes, and Load Balancing	Trelis Research	YouTube → Search 'vLLM Production Deployment Docker Kubernetes Load Balancing Trelis Research'	Containerise vLLM, deploy to Kubernetes, add a load balancer — production-grade setup.

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
9	Paper	Efficient Memory Management for Large Language Model Serving with PagedAttention	Kwon et al., 2023	arxiv.org → Search 'Efficient Memory Management Large Language Model Serving PagedAttention'	The original vLLM paper. Read the abstract and section 2 (Motivation). Understand KV cache waste.
10	Docs	vLLM Official Documentation — Quickstart + Performance Tuning	vLLM Team	docs.vllm.ai — read Quickstart + Performance + Supported Models + Quantisation	Excellent docs. Read Quickstart + Performance Tuning completely before deploying in production.

3.2 — vLLM Core Concepts

PagedAttention — The Key Innovation

Traditional LLM inference: each request pre-allocates contiguous GPU memory for its KV cache (key-value attention cache). If a request is allocated for 2048 tokens but uses only 200, 90% of that memory is wasted — and other requests cannot use it. This is called memory fragmentation.

PagedAttention: inspired by virtual memory in operating systems — it manages KV cache in fixed-size 'pages' that are allocated on-demand and can be shared between requests (for beam search or prefix caching). Result: near-zero memory waste, 24x higher throughput.

- Memory utilisation: 95%+ vs ~30% with naive allocation
- Throughput: 24x higher than HuggingFace text-generation-inference baseline
- Continuous batching: new requests join the batch as soon as a sequence finishes — no waiting for the full batch to complete

vLLM Quick Start — Production Deployment

```
# Install vLLM (requires CUDA GPU, at least 16GB VRAM for 7B FP16)
pip install vllm

# Serve Llama 3.1 8B Instruct with OpenAI-compatible API
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-8B-Instruct \
  --dtype auto \                # auto-selects FP16 or BF16
  --api-key your-secret-key \
  --host 0.0.0.0 \
  --port 8000 \
  --max-model-len 8192 \        # context window
  --gpu-memory-utilization 0.90  # use 90% of VRAM

# With AWQ quantisation (for 70B on a single A100 80GB):
python -m vllm.entrypoints.openai.api_server \
  --model casperhansen/llama-3-70b-instruct-awq \
  --quantization awq \
  --dtype half \
```

```
  --gpu-memory-utilization 0.95

# Multi-GPU (tensor parallel across 2 GPUs):
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-70B-Instruct \
  --tensor-parallel-size 2 \    # split across 2 GPUs
  --dtype bfloat16

# Your OpenAI SDK code works with ONE change:
# client = OpenAI(base_url='http://vllm-server:8000/v1', api_key='your-key')
```

vLLM Performance Tuning Parameters

--gpu-memory-utilization	Fraction of GPU memory to use. Default: 0.90. Increase to 0.95 to squeeze more throughput. Never set to 1.0 — leaves no room for activations.
--max-num-batched-tokens	Max tokens to process in one batch. Default: 8192. Increase for higher throughput at cost of memory.
--max-num-seqs	Max concurrent sequences. Controls how many requests are batched together.
--tensor-parallel-size	Number of GPUs for tensor parallelism. Use 2 for 70B on 2×A100, 4 for larger models.
--quantization	'awq', 'gptq', 'squeezellm'. Use awq for best quality at 4-bit. Cannot be changed after startup.
--enable-prefix-caching	Cache KV state for repeated prefixes (system prompts). Huge speedup for RAG with shared context.
--max-model-len	Override the model's maximum context length. Reduce to save memory — most queries use <4096 tokens.

3.3 — Stop Conditions

- ☐ Run vLLM locally (or on Colab/Lambda) — serve Llama 3.1 8B with OpenAI-compatible API
- ☐ Benchmark: vLLM throughput vs naive HuggingFace pipeline — confirm >3x improvement on same hardware
- ☐ AWQ: serve a 13B or 70B model using AWQ quantisation — confirm it fits in available VRAM
- ☐ Multi-GPU: if you have access to 2 GPUs, run tensor-parallel serving — demonstrate correct output
- ☐ Offline batch inference: process 1000 documents through vLLM in batch mode — benchmark tokens/second
- ☐ Production config: vLLM behind Nginx reverse proxy with HTTPS and API key validation
- ☐ Prefix caching enabled: demonstrate reduced latency for requests sharing the same system prompt

TOPIC 4 — OpenAI-Compatible Servers (LiteLLM, LocalAI, LMStudio) · *Week 4 · One API to rule all models*

Every major self-hosted inference server now exposes an OpenAI-compatible API. This means your existing application code — any code using the OpenAI Python SDK — works unchanged against Ollama, vLLM, LiteLLM, LocalAI, and dozens of others. LiteLLM is the most important tool here: it provides a unified API for 100+ LLM providers.

4.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	LiteLLM Tutorial — One API for 100+ LLM Providers	Sam Witteveen	YouTube → Search 'LiteLLM Tutorial One API 100 LLM Providers Sam Witteveen'	Best LiteLLM tutorial. Covers routing, fallbacks, cost tracking, and proxy server.
2	Video	LiteLLM Proxy Server — Deploy a Unified LLM Gateway	BerriAI (LiteLLM creators)	YouTube → BerriAI YouTube → Search 'LiteLLM Proxy Server Deploy Unified LLM Gateway'	Official tutorial for the LiteLLM proxy — the production gateway for multi-model routing.
3	Video	LocalAI — Run Any AI Model Locally (OpenAI API Compatible)	Mervin Praison	YouTube → Search 'LocalAI Run AI Model Locally OpenAI API Compatible Mervin Praison'	LocalAI goes beyond LLMs — runs Whisper, Stable Diffusion, and embeddings locally.
4	Video	LM Studio — Local AI Server for Developers (Full Guide)	Matt Williams (Ollama creator)	YouTube → Search 'LM Studio Local AI Server Developers Full Guide Matt Williams'	LM Studio for GUI-based local LLM management — good for non-technical team members.
5	Video	LiteLLM Cost Tracking and Rate Limiting — Production Setup	BerriAI (LiteLLM creators)	YouTube → BerriAI YouTube → Search 'LiteLLM Cost Tracking Rate Limiting Production'	Track costs per user, per API key — with rate limits per tier. Enterprise-ready.
6	Docs	LiteLLM Documentation — Proxy + Completion + Provider Pages	BerriAI Team	litellm.vercel.app/docs — read: Quick Start, Proxy, Supported Providers, Cost Tracking	Excellent short docs. Read Quick Start + Proxy + Cost Tracking sections.

4.2 — LiteLLM: The Production Multi-Model Gateway

Why LiteLLM Changes Everything

```
# Without LiteLLM: different SDK calls for each provider
from openai import OpenAI
from anthropic import Anthropic
import google.generativeai as genai
```

```
# Each provider: different method signatures, different error types,
# different streaming formats, different retry patterns... nightmare

# ──────────────────────────────────────────────────────────────────

# WITH LiteLLM: one function call, any provider
from litellm import acompletion

# OpenAI
await acompletion(model='gpt-4o', messages=[...])

# Anthropic — identical call
await acompletion(model='claude-3-5-sonnet-20241022', messages=[...])

# Google
await acompletion(model='gemini/gemini-1.5-pro', messages=[...])

# Your self-hosted Ollama server
await acompletion(model='ollama/llama3.1', messages=[...],
                  api_base='http://your-server:11434')

# Your vLLM server
await acompletion(model='hosted_vllm/llama3.1',
                  api_base='http://your-vllm:8000', messages=[...])

# One function. Any model. Any provider.
# Automatic fallback: if primary fails, try backup
await acompletion(
    model='gpt-4o',
    messages=[...],
    fallbacks=['claude-3-5-sonnet-20241022', 'ollama/llama3.1'],
)
```

LiteLLM Proxy — The Production Gateway

- Deploy LiteLLM as a centralized proxy server — all your services call one endpoint
- Virtual API keys: create per-user or per-service keys, each with their own rate limits and budget
- Model aliases: call 'my-fast-model' and LiteLLM routes to the actual model — change routing without code changes
- Automatic fallback: if OpenAI fails, LiteLLM automatically routes to Anthropic or your local Ollama
- Cost tracking: every call logged with exact cost — exportable to Prometheus/Grafana

> The LiteLLM proxy is one of the most powerful tools in your production AI stack. Deploy it as a sidecar service. Every other service (Fastify, FastAPI, workers) calls only the LiteLLM proxy — not individual model APIs. When you want to switch models or add a fallback, you change the LiteLLM config file, not your application code.

4.3 — Stop Conditions

☐ LiteLLM basic: call GPT-4o, Claude 3.5 Sonnet, and your Ollama server with the identical acompletion() call
☐ LiteLLM fallback: configure OpenAI → Anthropic → Ollama fallback chain — test by disabling each in turn
☐ LiteLLM proxy deployed: all services route through the proxy — individual provider API keys not in app code
☐ Virtual API keys: create a 'free-tier' key limited to 100 req/day and a 'pro-tier' key with unlimited access
☐ Cost tracking: LiteLLM proxy logs show per-model cost breakdown — matches actual OpenAI invoice

TOPIC 5 — Ray Serve: Scalable AI Model Serving · *Weeks 5–6 · Autoscaling AI microservices on Kubernetes*

Ray Serve is the production model serving framework from Anyscale — the team behind vLLM and Ray. It handles autoscaling, traffic routing, canary deployments, multi-model pipelines, and GPU scheduling. It is the serving framework used by many of the largest AI deployments in production.

5.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	Introduction to Ray Serve — Official Tutorial (YouTube + Docs)	Anyscale (official channel)	YouTube → Anyscale YouTube → Search 'Introduction Ray Serve Official Tutorial'	Official Ray Serve intro from Anyscale. Watch the full series — 4 parts.
2	Video	Ray Serve — Deploy Machine Learning Models at Scale	James Briggs	YouTube → Search 'Ray Serve Deploy Machine Learning Models Scale James Briggs'	Practical Ray Serve deployment — from FastAPI model to autoscaling endpoint.
3	Video	Ray Serve LLM Deployment — Serve vLLM with Ray Serve	Anyscale (official channel)	YouTube → Anyscale YouTube → Search 'Ray Serve LLM Deployment Serve vLLM'	Combine vLLM (inference engine) with Ray Serve (scaling) — the production ML serving stack.
4	Video	Ray Serve Autoscaling — Scale to Zero and Scale Up on Traffic	Anyscale (official channel)	YouTube → Anyscale YouTube → Search 'Ray Serve Autoscaling Scale Zero Traffic'	Scale to zero (no cost when idle), scale up in seconds when traffic arrives.
5	Video	Ray Serve A/B Testing and Canary Deployments	Anyscale (official channel)	YouTube → Anyscale YouTube → Search 'Ray Serve AB Testing Canary Deployments'	Route 10% of traffic to a new model version before full rollout — production deployment pattern.
6	Video	Ray Serve Multi-Model Pipeline — Chaining Models with Ray	James Briggs	YouTube → Search 'Ray Serve Multi-Model Pipeline Chaining Models James Briggs'	Chain multiple models (embed → classify → generate) in a single autoscaling pipeline.
7	Video	Ray on Kubernetes — Deploy Ray Cluster with KubeRay Operator	Anyscale (official channel)	YouTube → Anyscale YouTube → Search 'Ray Kubernetes Deploy Cluster KubeRay Operator'	Production: Ray on Kubernetes using the KubeRay operator — the enterprise deployment path.
8	Docs	Ray Serve Documentation — Key Concepts and Production Guide	Anyscale / Ray Team	docs.ray.io/en/latest/serve — read: Getting Started + Key Concepts + Production Guide	Good docs. Read Getting Started + Key Concepts before writing any Ray Serve code.

5.2 — Ray Serve Core Concepts

Deployments — The Unit of Scale

```
from ray import serve
from fastapi import FastAPI
from vllm import LLM, SamplingParams

app = FastAPI()

@serve.deployment(
    # Autoscaling config
    autoscaling_config={
        'min_replicas': 0,           # scale to zero when idle
        'max_replicas': 4,           # max 4 replicas under load
        'target_num_ongoing_requests_per_replica': 10,
    },
    # Resource requirements per replica
    ray_actor_options={
        'num_gpus': 1,               # each replica gets 1 GPU
        'num_cpus': 4,
    },
    # Health check
    health_check_period_s=30,
    health_check_timeout_s=60,
)
@serve.ingress(app)
class LLMDeployment:
    def __init__(self):
        # Model loaded once per replica, not per request
        self.llm = LLM(model='meta-llama/Llama-3.1-8B-Instruct',
                       gpu_memory_utilization=0.90)

    @app.post('/v1/chat/completions')
    async def chat(self, request: dict):
        params = SamplingParams(temperature=0.7, max_tokens=512)
        outputs = self.llm.generate([request['messages'][-1]['content']], params)
        return {'choices': [{'message': {'content': outputs[0].outputs[0].text}}]}

# Deploy
serve.run(LLMDeployment.bind())
```

Multi-Model Pipeline — The Ray Serve Superpower

- Stage 1 Deployment: EmbedModel — generates embeddings (CPU-only, many replicas)
- Stage 2 Deployment: RAGRetriever — queries Qdrant (CPU, I/O bound, many replicas)
- Stage 3 Deployment: GenerateModel — runs the LLM (GPU, fewer replicas)
- Each stage scales independently — CPU stages scale to 20 replicas, GPU stages scale to 4
- No bottleneck: the GPU stage is never waiting for the embedding stage — they scale separately

This is the key insight of Ray Serve for AI pipelines: different stages have different resource requirements and different scaling profiles. Embedding is cheap and fast — scale it to 50 replicas. LLM inference is expensive and slow — scale it to 4 GPU replicas. Ray Serve manages this automatically.

5.3 — Stop Conditions

- ☐ Deploy a FastAPI model serving endpoint with Ray Serve — autoscaling from 0 to 3 replicas under load
- ☐ LLM deployment: Ray Serve wraps vLLM — OpenAI-compatible endpoint with autoscaling
- ☐ Multi-model pipeline: embedding + retrieval + generation as 3 separate Ray Serve deployments
- ☐ Canary deployment: 10% of traffic routed to new model version — 90% to stable version
- ☐ Scale to zero: verify deployment scales to 0 replicas after 5 minutes of idle — no cost
- ☐ Load test: use locust to send 100 concurrent requests — Ray Serve autoscales and handles all

TOPIC 6 — Modal: Serverless GPU Inference · *Week 7 · Run any model on any GPU in 10 lines of Python*

Modal is the most developer-friendly serverless GPU platform. You write a Python function, decorate it with @modal.function, and Modal handles container builds, GPU scheduling, cold starts, autoscaling, and billing. No Kubernetes, no Docker registry, no GPU driver configuration. Just Python. It is the fastest path from 'I have a model' to 'I have a production endpoint'.

6.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Modal Tutorial — Run Python on GPUs in the Cloud (Full Walkthrough)	Weights & Biases (official channel)	YouTube → W&B YouTube → Search 'Modal Tutorial Run Python GPUs Cloud Full Walkthrough'	Best Modal intro tutorial. Covers functions, classes, volumes, schedules.
2	Video	Modal for LLM Inference — Deploy vLLM on Modal Serverless GPU	Modal (official channel)	YouTube → Modal official YouTube → Search 'Modal LLM Inference Deploy vLLM Serverless GPU'	Official Modal tutorial for LLM serving with vLLM — the recommended pattern.
3	Video	Modal vs AWS Lambda vs Google Cloud Run — Serverless Compute Comparison	Fireship	YouTube → Search 'Modal vs AWS Lambda Cloud Run Serverless Compute Comparison Fireship'	When to use Modal vs traditional serverless — GPU makes Modal uniquely powerful.
4	Video	Modal Web Endpoints — Serve FastAPI Apps on Serverless GPUs	Modal (official channel)	YouTube → Modal official YouTube → Search 'Modal Web Endpoints Serve FastAPI Apps Serverless GPUs'	Expose your Modal functions as HTTP endpoints — no infrastructure management needed.
5	Video	Modal Volumes and Caching — Persist Model Weights Across Cold Starts	Modal (official channel)	YouTube → Modal official YouTube → Search 'Modal Volumes Caching Persist Model Weights Cold Starts'	Modal Volumes: cache your model weights — avoid re-downloading 14GB on every cold start.
6	Video	Fine-Tuning on Modal — Run QLoRA Training Jobs on GPU	Weights & Biases (official channel)	YouTube → W&B YouTube → Search 'Fine-Tuning Modal Run QLoRA Training Jobs GPU'	Use Modal for Phase 4 fine-tuning — cheaper and simpler than managing your own GPU server.
7	Video	Modal Schedules — Run Batch AI Jobs on a Schedule	Modal (official channel)	YouTube → Modal official YouTube → Search 'Modal Schedules Run Batch AI Jobs Schedule'	Schedule your Prefect-like pipelines on Modal GPU — nightly document embedding runs.
8	Docs	Modal Documentation — Guide: Getting Started + Web Endpoints + GPUs	Modal Team	modal.com/docs — read Getting Started + Web Endpoints + Volumes + GPU guide	Excellent short docs. The Getting Started tutorial takes 15 minutes — do it before watching videos.

6.2 — Modal Patterns

Pattern 1: Serve vLLM on Modal (Production Endpoint)

```
import modal

# Define the container image — built once, cached by Modal
image = (modal.Image.debian_slim()
    .pip_install('vllm', 'fastapi')
    .env({'HF_HUB_ENABLE_HF_TRANSFER': '1'})  # fast HF downloads
)

app = modal.App('llm-serving')

# @modal.cls — class-based deployment — model loaded once, reused
@app.cls(
    image=image,
    gpu=modal.gpu.A10G(),      # A10G GPU (~$0.59/hr, only charged when running)
    container_idle_timeout=300, # keep alive 5 min after last request
    volumes={'/models': modal.Volume.from_name('model-weights',
create_if_missing=True)},
)
class LLMEndpoint:
    model_name: str = 'meta-llama/Llama-3.1-8B-Instruct'

    @modal.enter()  # runs once when container starts (cold start)
    def load_model(self):
        from vllm import LLM
        self.engine = LLM(
            model=self.model_name,
            download_dir='/models',   # cached in Modal Volume
            gpu_memory_utilization=0.90,
        )
        print('Model loaded!')

    @modal.web_endpoint(method='POST')  # HTTP endpoint
    async def generate(self, request: dict):
        from vllm import SamplingParams
        params = SamplingParams(temperature=0.7, max_tokens=512)
        prompt = request['messages'][-1]['content']
        output = self.engine.generate([prompt], params)[0]
        return {'response': output.outputs[0].text}

# Deploy: modal deploy llm_serving.py
# You get a URL like: https://your-workspace--llm-serving-llmendpoint-
generate.modal.run
# Cost: $0 when idle, $0.59/hr when handling requests
```

Pattern 2: Modal Volume for Model Caching

- Problem: every cold start downloads the 14GB model from HuggingFace — takes 3–5 minutes
- Solution: Modal Volume persists the model weights between container restarts
- First cold start: downloads model to Volume — 3 minutes
- All subsequent cold starts: model already in Volume — 15–30 seconds
- Implementation: mount Volume at /models, pass download_dir='/models' to vLLM

Pattern 3: Batch Processing on Modal GPU

```
# Run nightly document embedding batch on Modal GPU
# Cheaper than keeping a GPU server running 24/7

@app.function(
    image=image,
    gpu=modal.gpu.T4(),              # cheaper T4 for batch work
    schedule=modal.Cron('0 2 * * *'), # run at 2am every night
    timeout=3600,                    # 1 hour max runtime
)
def nightly_embedding_job():
    from sentence_transformers import SentenceTransformer
    import psycopg2

    model = SentenceTransformer('BAAI/bge-small-en-v1.5')

    # Fetch unembedded documents from PostgreSQL
    conn = psycopg2.connect(os.environ['DATABASE_URL'])
    # ... process and embed ...

    print(f'Embedded {count} documents in {elapsed:.1f}s')

# Cost: T4 GPU = $0.22/hr
# A 1-hour embedding job = $0.22/night = ~$6.60/month
# Vs always-on T4 server = $0.22 * 24 * 30 = $158/month
# Savings: 97%
```

6.3 — Stop Conditions

☐ Modal hello-world: deploy a simple function to Modal and call it via HTTP — 10 lines of Python
☐ LLM endpoint: vLLM served on Modal A10G — generate completions via HTTP POST
☐ Model caching with Volume: second cold start takes <30 seconds (model cached, not re-downloaded)
☐ Scheduled job: nightly embedding batch runs automatically at 2am — confirmed in Modal dashboard
☐ Cost calculation: compare Modal pay-per-use vs always-on GPU server for your specific workload

- ☐ Fine-tuning on Modal: run a QLoRA fine-tuning job from Phase 4 on Modal GPU — no local GPU needed

TOPIC 7 — Replicate: Model API Marketplace · *Week 7 · Run 100,000+ models via API — including your own*

Replicate is the model API marketplace — run any open-source model (Llama, Stable Diffusion, Whisper, SDXL, CodeLlama) via a simple API without managing any infrastructure. You can also push your own fine-tuned models to Replicate and share or monetise them. For multi-modal AI features and image generation, Replicate is often simpler and cheaper than other options.

7.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Replicate Tutorial — Run AI Models via API (Full Introduction)	Replicate (official channel)	YouTube → Replicate official YouTube → Search 'Replicate Tutorial Run AI Models API Introduction'	Official Replicate tutorial. Covers the UI, Python client, API, and model versioning.
2	Video	Run Stable Diffusion XL on Replicate — Image Generation Tutorial	David Ondrej	YouTube → Search 'Run Stable Diffusion XL Replicate Image Generation Tutorial David Ondrej'	SDXL on Replicate — the cheapest way to add production image generation to your app.
3	Video	Push Your Own Model to Replicate — Deploy Fine-Tuned Models	Replicate (official channel)	YouTube → Replicate official YouTube → Search 'Push Own Model Replicate Deploy Fine-Tuned'	Monetise your fine-tuned models — push to Replicate, set a price, share the API.
4	Video	Replicate Streaming — Stream LLM Output Token by Token	Replicate (official channel)	YouTube → Replicate official YouTube → Search 'Replicate Streaming LLM Output Token'	Stream Llama or Mistral outputs token by token via Replicate API.
5	Video	Replicate vs Hugging Face Inference Endpoints — Which to Use	Sam Witteveen	YouTube → Search 'Replicate vs HuggingFace Inference Endpoints Which to Use Sam Witteveen'	Key comparison — Replicate for multi-modal/image models, HF Endpoints for text LLMs.
6	Docs	Replicate Python Client Documentation	Replicate Team	replicate.com/docs/get-started/python — read completely (short, 20 minutes)	Short, excellent docs. Read the complete Python client guide before writing code.

7.2 — Replicate Best Use Cases

Image generation	SDXL, FLUX.1, Stable Diffusion 3 — generate product images, UI mockups, marketing visuals. ~$0.0023/image.
Image editing	InstructPix2Pix, ControlNet — edit images based on text instructions. Unique Replicate strength.
Video generation	Stable Video Diffusion, AnimateDiff — generate short videos from images or text.

Audio transcription	Incredibly Fast Whisper (faster-whisper on Replicate) — transcribe at 60x realtime speed. ~$0.001/min.
Your fine-tuned models	Push your Phase 4 fine-tuned Llama to Replicate — instant API endpoint, pay per inference second.
Experimental models	Try new research models (LLaVA, Mistral, Qwen2-VL) without setting up serving infrastructure.

```
import replicate

# Generate an image with SDXL
output = replicate.run(
    'stability-
ai/sdxl:39ed52f2a78e934b3ba6e2a89f5b1c712de7dfea535525255b1aa35c5565e08b',
    input={'prompt': 'A professional product photo of a SaaS dashboard, clean,
minimal'},
)
image_url = output[0]  # URL to generated image

# Transcribe audio with faster-whisper
transcription = replicate.run(
    'vaibhavs10/incredibly-fast-
whisper:3ab86df6c8f54c11309d4d1f930ac292bad43ace52d10c80d87eb258b3c9f79c',
    input={'audio': open('meeting.mp3', 'rb'), 'language': 'en'},
)
text = transcription['text']

# Stream Llama 3.1 output
for event in replicate.stream(
    'meta/meta-llama-3.1-8b-instruct',
    input={'prompt': 'Explain RAG in 3 sentences', 'max_tokens': 200},
):
    print(str(event), end='', flush=True)
```

7.3 — Stop Conditions

- ☐ Image generation: SDXL generates a product image in your AI SaaS app — user sees result in <10 seconds
- ☐ Audio transcription: user uploads a 10-minute audio file — transcribed via Replicate faster-whisper in <30 seconds
- ☐ Push to Replicate: your Phase 4 fine-tuned model deployed on Replicate — API endpoint working
- ☐ Streaming: Llama output on Replicate streams token-by-token to your frontend

TOPIC 8 — BentoML: Package and Deploy ML Models · *Week 8 · The ML model packaging standard*

BentoML standardises how you package, version, and deploy machine learning models. A 'Bento' is a self-contained unit containing the model weights, the serving code, the dependencies, and the API definition — like a Docker image but specifically designed for ML models. BentoCloud provides managed serving for Bentos.

8.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	BentoML Tutorial — Package and Deploy ML Models (Full Introduction)	BentoML (official channel)	YouTube → BentoML official YouTube → Search 'BentoML Tutorial Package Deploy ML Models Introduction'	Official BentoML intro — covers Service API, Runners, and Bento packaging.
2	Video	BentoML with HuggingFace — Deploy Transformers Models as REST APIs	BentoML (official channel)	YouTube → BentoML official YouTube → Search 'BentoML HuggingFace Deploy Transformers REST APIs'	Deploy any HuggingFace model as a REST API — the most common BentoML use case.
3	Video	BentoML vs FastAPI — When to Use Each for ML Serving	James Briggs	YouTube → Search 'BentoML vs FastAPI When Use Each ML Serving James Briggs'	Decision video: use FastAPI for custom logic, BentoML when packaging + standardisation matters.
4	Video	BentoCloud — Deploy Bentos to the Cloud with Autoscaling	BentoML (official channel)	YouTube → BentoML official YouTube → Search 'BentoCloud Deploy Bentos Cloud Autoscaling'	BentoCloud managed serving — zero-config autoscaling for your Bento.
5	Docs	BentoML Documentation — Quickstart and Build a Service	BentoML Team	docs.bentoml.com — read Quickstart + Build a Service + Deploy to BentoCloud	Read Quickstart + Build a Service. BentoML docs are clear and practical.

8.2 — BentoML Core Concepts

```
# bentofile.yaml — describes the Bento package
service: 'service:LLMService'
python:
  packages:
    - vllm
    - fastapi
    - sentence-transformers

# service.py — the serving code
import bentoml
from vllm import LLM, SamplingParams
```

```
@bentoml.service(
    resources={'gpu': 1, 'memory': '16Gi'},
    traffic={'timeout': 60},
)
class LLMService:
    def __init__(self) -> None:
        self.engine = LLM(model='meta-llama/Llama-3.1-8B-Instruct')

    @bentoml.api
    def generate(self, prompt: str) -> str:
        params = SamplingParams(temperature=0.7, max_tokens=512)
        output = self.engine.generate([prompt], params)[0]
        return output.outputs[0].text

# Build the Bento (packages model + code + deps into one artifact)
# bentoml build

# Serve locally
# bentoml serve service:LLMService

# Deploy to BentoCloud
# bentoml deploy -n llm-service
```

8.3 — Stop Conditions

- ☐ Build a Bento from a HuggingFace model — bento artifact contains model + serving code + dependencies
- ☐ Serve the Bento locally — OpenAPI docs auto-generated at /docs
- ☐ Version a Bento: build v1, build v2, rollback to v1 — all versions available in the registry
- ☐ Deploy to BentoCloud — endpoint live with autoscaling (or deploy to your own server via Docker)

TOPIC 9 — NVIDIA Triton Inference Server · *Week 9 · Enterprise-grade inference at maximum GPU utilisation*

NVIDIA Triton Inference Server is the gold standard for enterprise AI inference. It supports every major model format (PyTorch, TensorFlow, ONNX, TensorRT), provides dynamic batching, concurrent model execution, and detailed Prometheus metrics. It is more complex to set up than vLLM but offers the highest throughput for production deployments at scale.

9.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	Triton Inference Server — Full Beginner Tutorial Series (5 videos)	NVIDIA On-Demand (official channel)	YouTube → Search 'Triton Inference Server Full Beginner Tutorial Series NVIDIA' → open playlist	Official NVIDIA Triton tutorial series. Watch all 5 parts — the most complete introduction.
2	Video	Triton Inference Server — Getting Started with LLMs	NVIDIA On-Demand (official channel)	YouTube → NVIDIA YouTube → Search 'Triton Inference Server Getting Started LLMs'	Specifically covers LLM serving with Triton + TensorRT-LLM — the production NVIDIA stack.
3	Video	Triton Dynamic Batching — Maximise GPU Utilisation	NVIDIA On-Demand (official channel)	YouTube → NVIDIA YouTube → Search 'Triton Dynamic Batching Maximise GPU Utilisation'	Dynamic batching is Triton's key performance feature — understand it before benchmarking.
4	Video	TensorRT-LLM — NVIDIA's Fastest LLM Inference Engine	NVIDIA On-Demand (official channel)	YouTube → NVIDIA YouTube → Search 'TensorRT-LLM Fastest LLM Inference Engine NVIDIA'	TensorRT-LLM converts any model to optimised NVIDIA kernels — maximum NVIDIA GPU throughput.
5	Video	Triton vs vLLM — Choosing the Right LLM Inference Server	Trelis Research	YouTube → Search 'Triton vs vLLM Choosing Right LLM Inference Server Trelis Research'	Direct comparison — Triton excels for multi-model, multi-framework environments.
6	Docs	Triton Inference Server GitHub — Quick Start Guide	NVIDIA Team	github.com/triton-inference-server/server — read Quick Start + Tutorials + Architecture	Read the Quick Start and Architecture docs before attempting any hands-on work.

9.2 — When to Use Triton vs vLLM

Factor	vLLM	Triton + TensorRT-LLM
Setup complexity	Moderate — pip install, one command	High — model conversion, repository setup, config files
LLM throughput	Excellent (PagedAttention)	Highest possible on NVIDIA GPUs (TensorRT kernels)

Factor	vLLM	Triton + TensorRT-LLM
Multi-framework	LLMs only (PyTorch-based)	Any framework: PyTorch, TensorFlow, ONNX, TensorRT
Multi-model	One model per server instance	10+ models simultaneously, concurrent execution
Model formats	HuggingFace models	ONNX, TensorRT engines, PyTorch, TensorFlow SavedModel
Monitoring	Basic — Prometheus metrics	Rich — per-model latency, queue depth, GPU utilisation
Best for	LLM-only serving, most production use cases	Enterprise: vision + NLP + custom models on same infra
Team requirement	AI engineer can manage	MLOps engineer required for complex deployments

⍰ Triton is the correct choice when you need to serve multiple model types simultaneously — for example, an embedding model + a classification model + an LLM on the same GPU server. For LLM-only serving, vLLM is simpler and equally performant. Learn Triton conceptually — you will encounter it in enterprise AI teams.

9.3 — Stop Conditions

- ☐ Run Triton in Docker with the official NVIDIA Triton container — server starts successfully
- ☐ Deploy a simple PyTorch model to Triton — HTTP inference endpoint working
- ☐ Dynamic batching enabled — benchmark shows higher throughput vs single-request mode
- ☐ Prometheus metrics: scrape Triton metrics endpoint — GPU utilisation, request latency visible
- ☐ Explain when Triton is preferable to vLLM — give a concrete production scenario for each

🗓 Phase 6 — Week-by-Week Schedule (12 Weeks)

Phase 6 is infrastructure-heavy. Weeks 1–6 are tool-by-tool learning. Weeks 7–9 integrate everything into the capstone. Weeks 10–12 are deployment, benchmarking, and documentation. You need access to at least one GPU — Google Colab (free T4), Lambda Labs (~$0.50/hr A10G), or Modal (pay-per-use) all work.

Week	Topic Focus	Exact Videos to Watch	Hands-On Task	Done When...
Wk 1	Quantisation Deep Dive	Quantisation Explained GGUF GPTQ AWQ — Fahd Mirza GGUF Format Explained — Matt Williams AWQ Quantisation 4-Bit — Trelis Research	Download Q4_K_M GGUF model from HuggingFace Run inference with llama.cpp Quantise a model yourself: FP16 → Q4_K_M	Can quantise any model; Q4_K_M vs Q8_0 quality comparison documented
Wk 2	Ollama Production	Ollama Production Deploy Server GPU — Matt Williams Ollama REST API Build Applications — Matt Williams Ollama Docker Deployment — Mervin Praison	Deploy Ollama on a cloud GPU server (Lambda Labs) Configure OLLAMA_NUM_PARALLEL=4 Verify OpenAI SDK connects with only base_url change	Ollama live on GPU server; OpenAI SDK works; custom model deployed via Modelfile
Wk 3	vLLM Fundamentals	vLLM Explained PagedAttention — YouTube search vLLM Tutorial Deploy HuggingFace Model — Trelis Research vLLM with AWQ Quantisation — Fahd Mirza	Serve Llama 3.1 8B with vLLM on GPU (Colab/Lambda) Benchmark vLLM vs Ollama on same hardware Enable prefix caching	vLLM serving with OpenAI-compatible API; throughput benchmark shows >3x improvement vs naive
Wk 4	vLLM Advanced + LiteLLM	vLLM Production Deployment Docker Kubernetes — Trelis vLLM Multi-GPU Serving Tensor Parallelism — vLLM official LiteLLM Tutorial One API 100 Providers — Sam Witteveen	vLLM behind Nginx with API key auth LiteLLM unified gateway connecting OpenAI + Anthropic + Ollama + vLLM Provider fallback tested	LiteLLM proxy running; all 4 providers callable via single function; fallback confirmed
Wk 5	Ray Serve	Introduction Ray Serve Official Tutorial — Anyscale (all parts) Ray Serve LLM Deployment vLLM — Anyscale Ray Serve Autoscaling Scale Zero — Anyscale	Deploy a FastAPI model endpoint with Ray Serve Add autoscaling: min=0, max=4 replicas Load test with locust — confirm autoscale	Ray Serve endpoint autoscales from 0→3 under load; scales back to 0 when idle
Wk 6	Ray Serve Multi-Model + Modal	Ray Serve Multi-Model Pipeline — James Briggs Ray Serve AB Testing Canary — Anyscale Modal Tutorial Run Python GPUs Cloud — W&B	3-stage pipeline: embed + retrieve + generate as separate Ray deployments Modal hello-world: deploy simple function to Modal GPU	Multi-model pipeline working; each stage scales independently; Modal function deployed
Wk 7	Modal LLM + Replicate	Modal LLM Inference Deploy vLLM Serverless GPU — Modal official Modal Volumes Caching Persist Model Weights — Modal official Replicate Tutorial Run AI Models API — Replicate official	vLLM on Modal A10G with model caching in Volume Image generation via Replicate SDXL Audio transcription via Replicate faster-whisper	Modal LLM endpoint live; model cached (cold start <30s); Replicate features integrated in app
Wk 8	BentoML + Triton Intro	BentoML Tutorial Package Deploy ML Models — BentoML official BentoML HuggingFace Deploy Transformers — BentoML official	Build a Bento from a HuggingFace model Run Triton in Docker with a simple PyTorch model	Bento builds and serves locally; Triton running with dynamic batching enabled

Week	Topic Focus	Exact Videos to Watch	Hands-On Task	Done When...
		Triton Inference Server Beginner Tutorial — NVIDIA (Ep 1–3)	Enable Triton dynamic batching	
Wk 9	Capstone — Core Infrastructure Build	Review: vLLM + LiteLLM + Modal docs	Build inference cluster: - Primary: vLLM on GPU server for LLM inference - Fallback: Ollama on same server - Gateway: LiteLLM proxy - Serverless batch: Modal for embedding jobs	Full inference cluster operational; LiteLLM routes and falls back correctly
Wk 10	Capstone — Observability + Cost Dashboard	Review: Prometheus + Grafana setup Review: LiteLLM cost tracking docs	Prometheus scrapes vLLM + Ollama + LiteLLM metrics Grafana dashboard: GPU utilisation, req/s, p95 latency, cost/request Slack alert on high GPU temperature or error rate	Grafana dashboard shows live metrics; alert triggers correctly on test condition
Wk 11	Capstone — Load Testing + Tuning	Review: locust load testing docs vLLM Performance Tuning docs	Load test entire inference cluster with locust (100 concurrent users) Tune vLLM: gpu_memory_utilization, max_num_seqs Document: requests/second and cost/request at peak load	Cluster handles 100 concurrent users; p95 latency <3 seconds; cost/request calculated
Wk 12	Capstone — Deploy + Benchmark + Open Source	Review: Railway + Docker deploy guides	Deploy LiteLLM proxy on Railway Write README with: - Architecture diagram - Benchmark table (throughput, latency, cost) - One-command setup - Hosted demo endpoint	Live inference cluster with public endpoint; README with benchmark data; GitHub public

⍰ Phase 6 Capstone — Self-Hosted LLM Inference Cluster

This is Portfolio Project #6 and the most technically impressive infrastructure project in your portfolio. A production-grade self-hosted inference cluster — with vLLM for high-throughput serving, LiteLLM as the gateway, Ollama as fallback, Modal for serverless batch, Prometheus/Grafana monitoring, and a live public endpoint — demonstrates senior MLOps engineering capability.

Full Project Specification

Project Name	llm-inference-cluster (GitHub repo name)
Primary Serving	vLLM on GPU server — Llama 3.1 8B Instruct (FP16) + Llama 3.1 70B (AWQ 4-bit)
Embedding Serving	Ollama serving nomic-embed-text — OpenAI-compatible /v1/embeddings endpoint
Gateway	LiteLLM proxy — unified API, cost tracking, rate limiting, fallback chain
Fallback Chain	vLLM → Ollama → OpenAI API — transparent failover, zero downtime
Batch Processing	Modal serverless — nightly embedding jobs on T4 GPU, ~$0.22/night
Image Generation	Replicate SDXL endpoint — integrated into main application
Quantisation Options	GGUF Q4_K_M (Ollama), AWQ 4-bit (vLLM 70B), FP16 (vLLM 8B)
Monitoring	Prometheus + Grafana — GPU utilisation, req/s, p95 latency, cost/request, error rate
Deployment	Docker Compose local dev; GPU server (Lambda Labs or similar); LiteLLM on Railway

Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                        APPLICATION LAYER                             │
│  Your AI SaaS (Phase 1-5) — uses OpenAI SDK                          │
│  FastAPI AI Service  ·  Fastify Backend  ·  Next.js Frontend         │
└──────────────────────────┬───────────────────────────────────────────┘
                           │  All requests go to LiteLLM proxy
                           ▼
┌──────────────────────────────────────────────────────────────────────┐
│                     LITELLM PROXY GATEWAY (Railway)                  │
│  Virtual API keys  ·  Rate limiting  ·  Cost tracking                │
│  Model aliases     ·  Fallback routing  ·  Request logging           │
└──────────┬──────────────────┬──────────────────┬──────────────────────┘
           │                  │                  │
           ▼                  ▼                  ▼
  PRIMARY: vLLM          FALLBACK:          CLOUD FALLBACK:
  (GPU Server)           Ollama             OpenAI API
```

```
____________          _____________    _____________
Llama 3.1 8B FP16     Llama 3.1 8B     gpt-4o-mini
Llama 3.1 70B AWQ     nomic-embed-text text-embedding-3-small
~500 req/min          ~50 req/min      $0.15/M tokens
~$0.0002/req          ~$0.0003/req     $0.0005/req

PROMETHEUS METRICS SCRAPE ──────────────────────────┐
(vLLM /metrics · Ollama /metrics · LiteLLM /metrics) │
                                                     ▼
                                            GRAFANA DASHBOARD
                                            GPU util · req/s
                                            p95 latency · cost

MODAL SERVERLESS (async batch):
Nightly document embedding on T4 GPU → Qdrant
Fine-tuning jobs → HuggingFace Hub
Scheduled at 2am · ~$0.22/night

REPLICATE (external API):
SDXL image generation · faster-whisper audio
```

Benchmark Table — Must Be in README

Metric	vLLM (8B FP16)	Ollama (8B Q4_K_M)	OpenAI API (gpt-4o-mini)
Throughput	~500 req/min (A10G)	~50 req/min (A10G)	Rate limited to ~3500 req/min
P50 latency	~0.8s first token	~1.2s first token	~0.6s first token
P95 latency	~2.1s first token	~3.5s first token	~2.8s first token
Cost per request	~$0.0002	~$0.0003	~$0.0005
Cost per 1M tokens	~$0.18 (infra cost)	~$0.28 (infra cost)	$0.15 (API cost)
Data stays in VPC	Yes	Yes	No
Fine-tuned models	Yes — deploy instantly	Yes — via Modelfile	Only if trained via OpenAI
Setup complexity	High	Low	Trivial

> ⍰ The benchmark table in your README is what gets this project noticed. Numbers like '$0.0002 vs $0.0005 per request = 60% cost savings at scale' are immediately legible to engineering managers, CTOs, and hiring teams. Calculate the annualised savings at 1M requests/day and include it.

GitHub README Requirements

- Live endpoint: public LiteLLM proxy URL — testers can make API calls with a provided test key

- Benchmark table: throughput, latency, cost for each serving tier
- Architecture diagram (ASCII in README, same as above)
- 'docker compose up' starts vLLM + Ollama + LiteLLM proxy in one command (requires GPU)
- Grafana dashboard screenshot: GPU utilisation and request metrics visible
- Cost calculator section: 'At X requests/day, self-hosting saves $Y/month vs OpenAI'
- Supported models list: which models are deployed and their VRAM requirements
- Integration guide: how to point your existing OpenAI SDK app at this cluster (one env var)

Estimated Time Investment

Weeks 1–2 (Quantisation + Ollama Production)	~20 hrs — 8 hrs videos + 12 hrs setup on GPU server
Weeks 3–4 (vLLM + LiteLLM)	~20 hrs — 8 hrs videos + 12 hrs deployment + benchmarking
Weeks 5–6 (Ray Serve + Modal)	~18 hrs — 8 hrs tutorials + 10 hrs hands-on
Weeks 7–8 (Replicate + BentoML + Triton)	~16 hrs — 8 hrs videos + 8 hrs hands-on
Weeks 9–10 (Capstone core + monitoring)	~18 hrs — all building and configuration
Weeks 11–12 (Load testing + deploy + docs)	~14 hrs — load testing, tuning, README, open source
Total Phase 6	~106 hours over 12 weeks ≈ 2.5 hrs/weekday + 4 hrs/weekend
GPU cost estimate	Lambda Labs A10G at ~$0.54/hr for ~40 hours = ~$22 total GPU cost for the entire phase

Phase 6 Completion Check: You have completed Phase 6 when: (1) The LiteLLM proxy is live with a public test endpoint, (2) The benchmark table in the README shows real measured numbers (not estimates), (3) The Grafana dashboard screenshot shows live GPU and request metrics, (4) 'docker compose up' starts the full cluster in one command. This project demonstrates you can own production AI infrastructure — not just call APIs.

Phase 6's biggest time sink: GPU configuration issues. CUDA driver mismatches, out-of-memory errors, and Docker networking issues will eat your first few days. Budget 4–6 hours for environment setup before any model serving code. Use Lambda Labs or Modal for the first experiments — no local GPU driver fights.